



BERT Pretraining and Fine-tuning

Rasha Obeidat, PhD

NLP

Rasha Obeidat

Pretraining and Fine-tuning

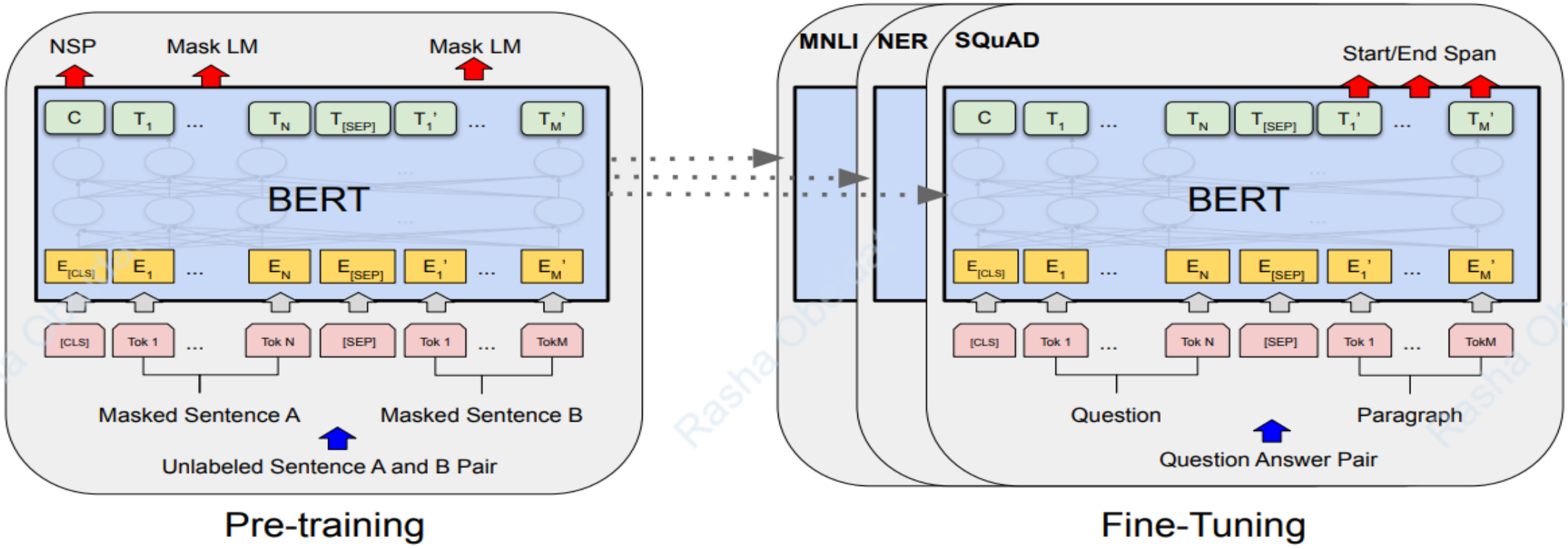


Figure 1: Overall pre-training and fine-tuning procedures for BERT. Apart from output layers, the same architectures are used in both pre-training and fine-tuning. The same pre-trained model parameters are used to initialize models for different down-stream tasks. During fine-tuning, all parameters are fine-tuned. [CLS] is a special symbol added in front of every input example, and [SEP] is a special separator token (e.g. separating questions/answers).

BERT Paper: https://aclanthology.org/N19-1423.pdf

BERT Pretraining and Fine-tuning

Pretraining:

- BERT is first trained on a **large unlabeled text corpus** .
- Learns general language representations through:
  - Masked Language Modeling (MLM)
  - Next Sentence Prediction (NSP)

Fine-tuning:

- BERT is adapted to **specific downstream tasks**.
- A task-specific layer is added (e.g., classification head).
- All BERT parameters are **updated** on the labeled dataset.
- The added task-specific classification layer(s)are trained from scratch.

Pretraining = general language knowledge
Fine-tuning = specializing to a task

Dataset for BERT Pretraining

Original BERT (English):

- BooksCorpus (800M words)
- English Wikipedia (2,500M words)

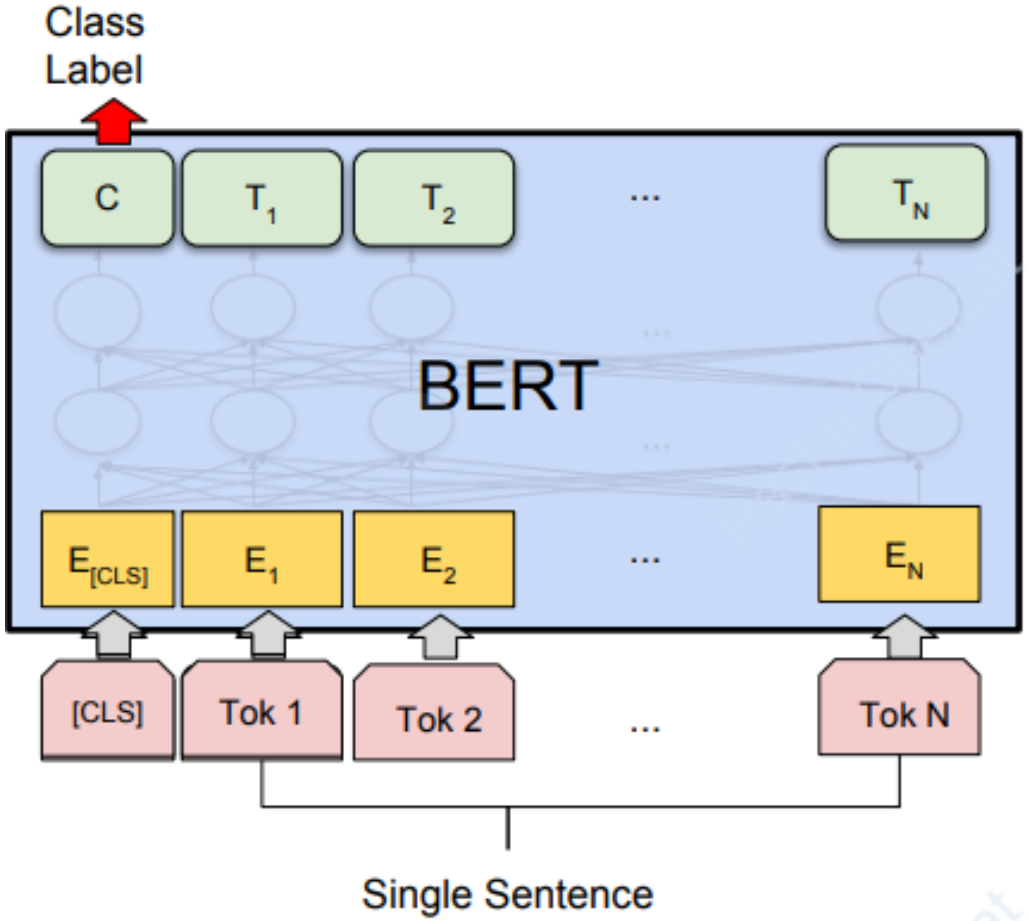
AraBERT (Arabic): (https://aclanthology.org/2020.osact-1.2.pdf)

- Arabic Wikipedia
- OSIAN corpus (Open Source International Arabic News) , consists of 3.5 million articles (~1B tokens)
- .5 billion words Arabic Corpus
- Arabic news websites and tweets

Dr. Rasha Obeidat ®

Fine-tuning BERT for Single Sentence Classification

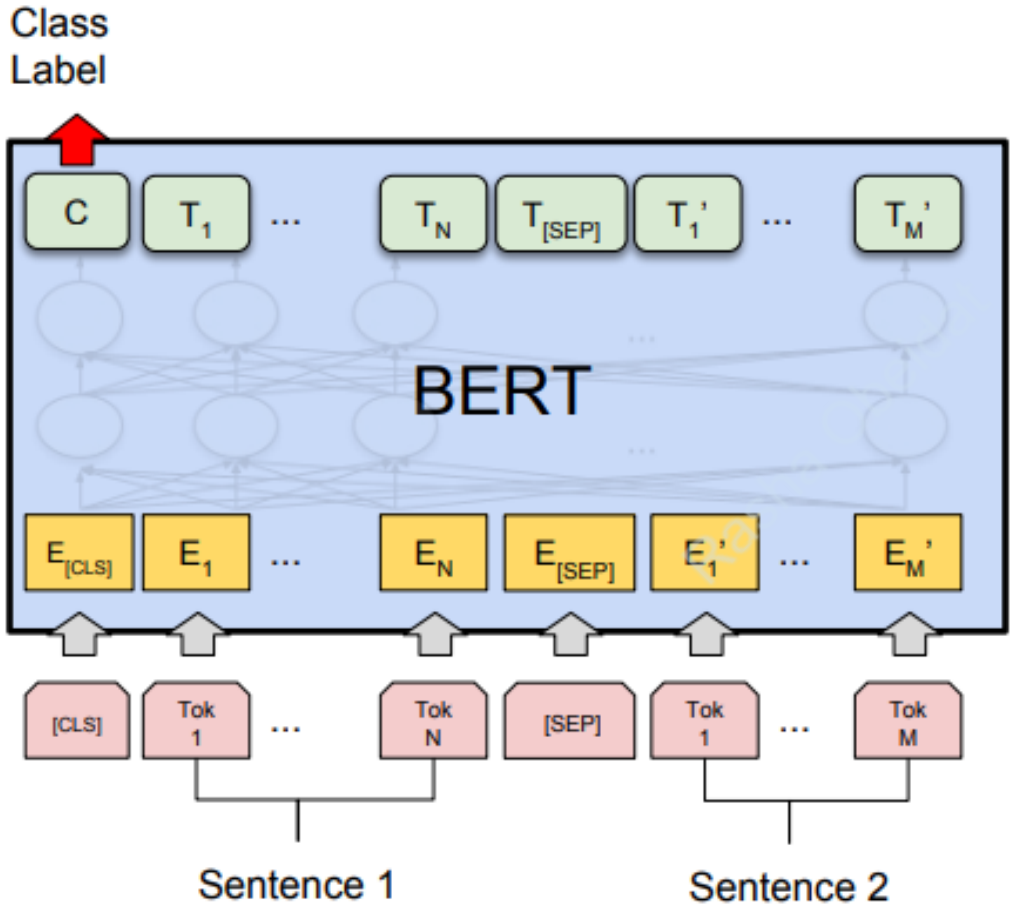
- BERT processes the input sentence and produces a contextualized embedding for each token.
- The final hidden state corresponding to the special [CLS] token is treated as the **summary representation** of the entire input.
- The [CLS] vector is then passed through a simple feedforward neural network (FFNN) followed by a Softmax layer to predict the **class label**.
- **Example tasks**
  - Sentiment Analysis
  - Spam Detection
  - Emotion Recognition
  - Topic Classification
  - Hate Speech Detection



BERT Paper: https://aclanthology.org/N19-1423.pdf

Fine-tuning BERT for Sentence Pair Classification

- BERT processes both sentences jointly as a single input, separated by the special [SEP] token.
- Each token receives a contextualized embedding based on both sentences together.
- The final hidden state corresponding to the special [CLS] token is used as the summary representation of the entire input pair.
- The [CLS] vector is passed through a simple feedforward neural network (FFNN) followed by a Softmax layer to predict the class label.
- **Example tasks**:
  - Natural Language Inference (NLI)
  - Paraphrase Detection
  - Question Answering (short answers)
  - Duplicate Question Detection



BERT Paper: https://aclanthology.org/N19-1423.pdf

Fine-tuning BERT for Sentence Pair Classification

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	**Average** -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
$\text{BERT}_{\text{BASE}}$	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
$\text{BERT}_{\text{LARGE}}$	**86.7/85.9**	**72.1**	**92.7**	**94.9**	**60.5**	**86.5**	**89.3**	**70.1**	**82.1**

Table 1: GLUE Test results, scored by the evaluation server (https://gluebenchmark.com/leaderboard). The number below each task denotes the number of training examples. The "Average" column is slightly different than the official GLUE score, since we exclude the problematic WNLI set.[8] BERT and OpenAI GPT are single-model, single task. F1 scores are reported for QQP and MRPC, Spearman correlations are reported for STS-B, and accuracy scores are reported for the other tasks. We exclude entries that use BERT as one of their components.

BERT Paper: https://aclanthology.org/N19-1423.pdf

Fine-tuning BERT for Sentence Pair Classification

Table 6 The overall performance of the examined baseline models and the human evaluator was measured using accuracy, macro-averaged precision, recall and F1 score

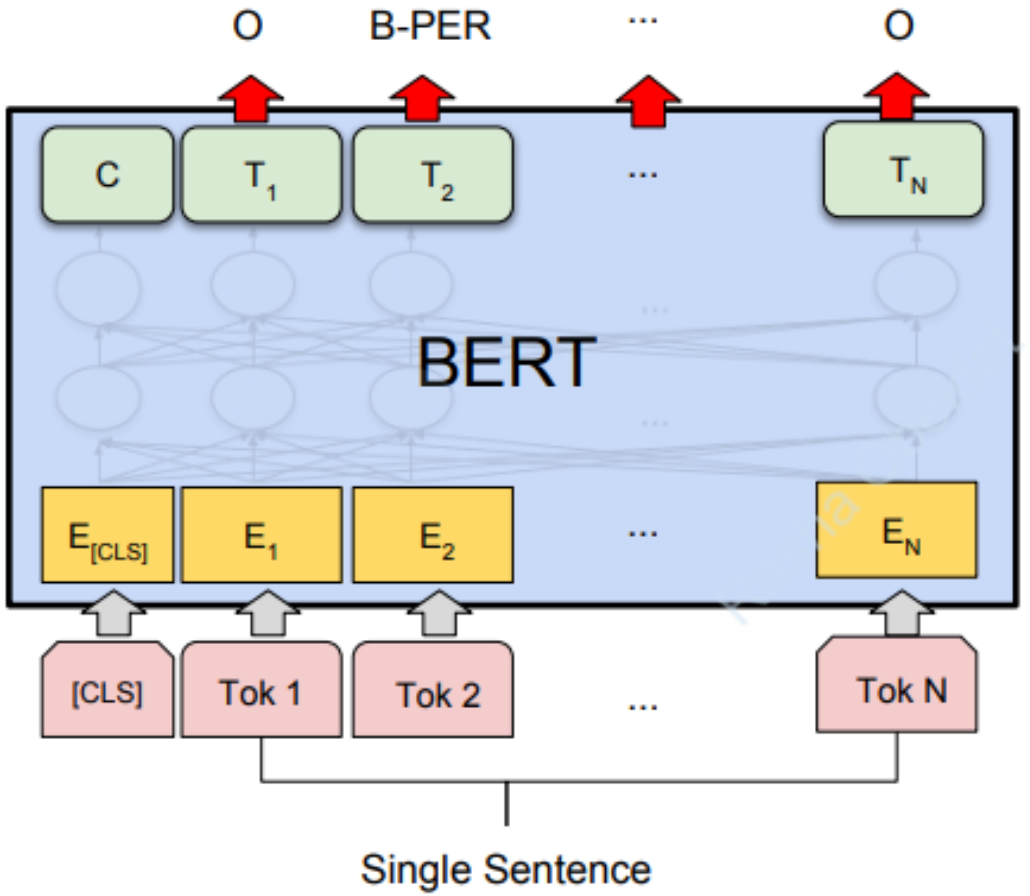
Model	Accuracy (%)	Macro-averaged		
		Precision (x)	Recall (%)	F1 score (%)
AraBERT-base v1	88.40	88.72	88.40	88.37
AraBERT-base v2	93.00	93.08	92.90	92.99
XML-R-ar	83	83.02	83	82.99
mBERT-cased-ar	81.6	81.61	81.6	81.59
mBERT-uncased-ar	82.2	82.25	82.2	82.19
ArDR-BiLSTM	73	75.55	73	72.30
GPT- ZSL	65.7	73.10	65.7	62.71
GPT -FSL	74.66	75.35	74.66	74.48
Human evaluation	95.6	95.64	95.6	95.59

https://link.springer.com/article/10.1007/s10579-024-09731-1

Obeidat, Rasha, et al. "ArEntail: manually-curated Arabic natural language inference dataset from news headlines." *Language Resources and Evaluation* (2024): 1-27.

Fine-tuning BERT for Sentence Tagging Tasks

- BERT processes the input sentence and produces a contextualized embedding for each token.
- Each token's final hidden state is used to predict a **label** independently (e.g., part of speech, named entity type).
- A simple classification head (linear layer + Softmax) is applied **separately** at each token's representation.
- The [CLS] token is ignored during tagging — only real tokens are labeled.
- During training, the loss is usually summed or averaged over all real tokens (excluding [CLS] and [SEP]).
- **Example tasks**:
  - Named Entity Recognition (NER)
  - Part of Speech (POS) Tagging
  - Chunking (shallow parsing)
    - E.g., "the big dog"->Noun phrase



BERT Paper: https://aclanthology.org/N19-1423.pdf

Fine-tuning BERT for Sentence Tagging Tasks

CSE: BiLSTM-CRF + character-level language model

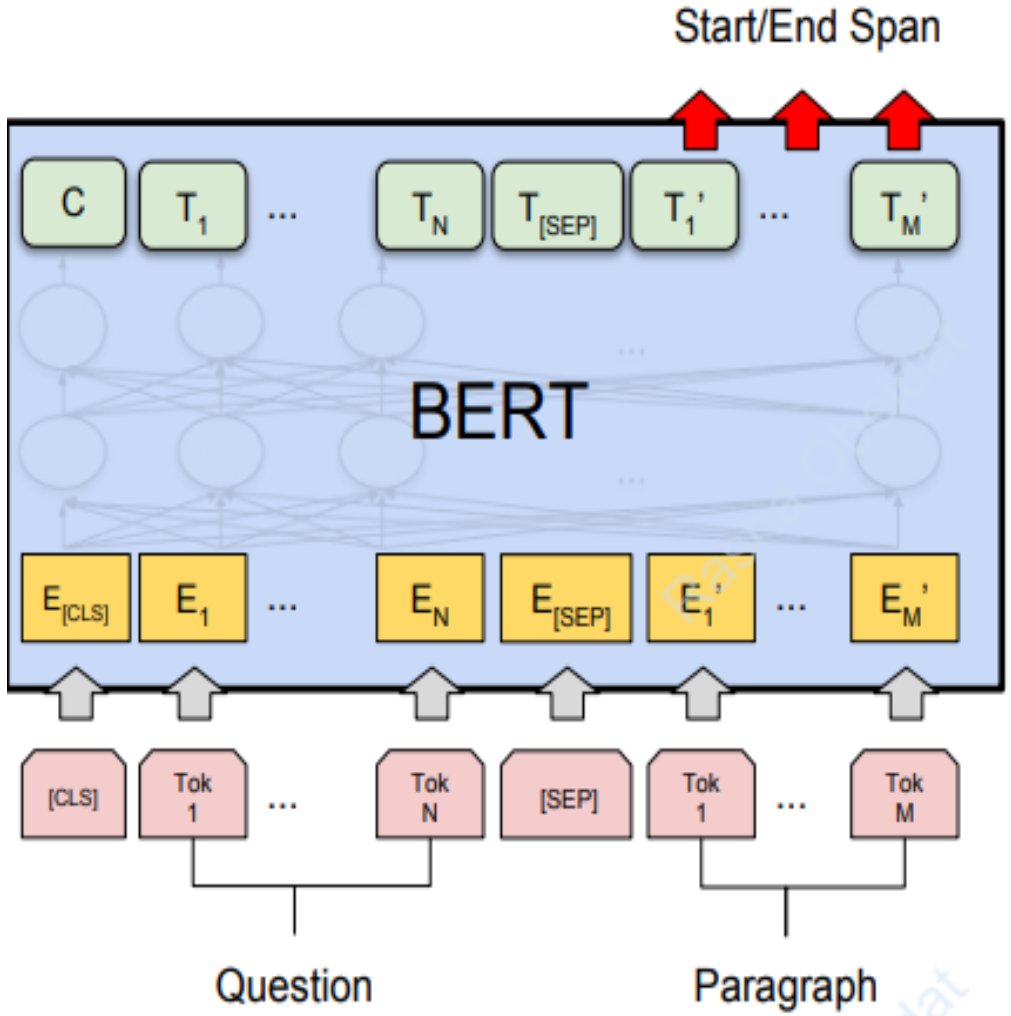
System	Dev F1	Test F1
ELMo (Peters et al., 2018a)	95.7	92.2
CVT (Clark et al., 2018)	-	92.6
CSE (Akbik et al., 2018)	-	**93.1**
Fine-tuning approach		
$\text{BERT}_{\text{LARGE}}$	96.6	92.8
$\text{BERT}_{\text{BASE}}$	96.4	92.4
Feature-based approach ($\text{BERT}_{\text{BASE}}$)		
Embeddings	91.0	-
Second-to-Last Hidden	95.6	-
Last Hidden	94.9	-
Weighted Sum Last Four Hidden	95.9	-
Concat Last Four Hidden	96.1	-
Weighted Sum All 12 Layers	95.5	-

Table 7: CoNLL-2003 Named Entity Recognition

BERT Paper: https://aclanthology.org/N19-1423.pdf

Fine-tuning BERT for MRC

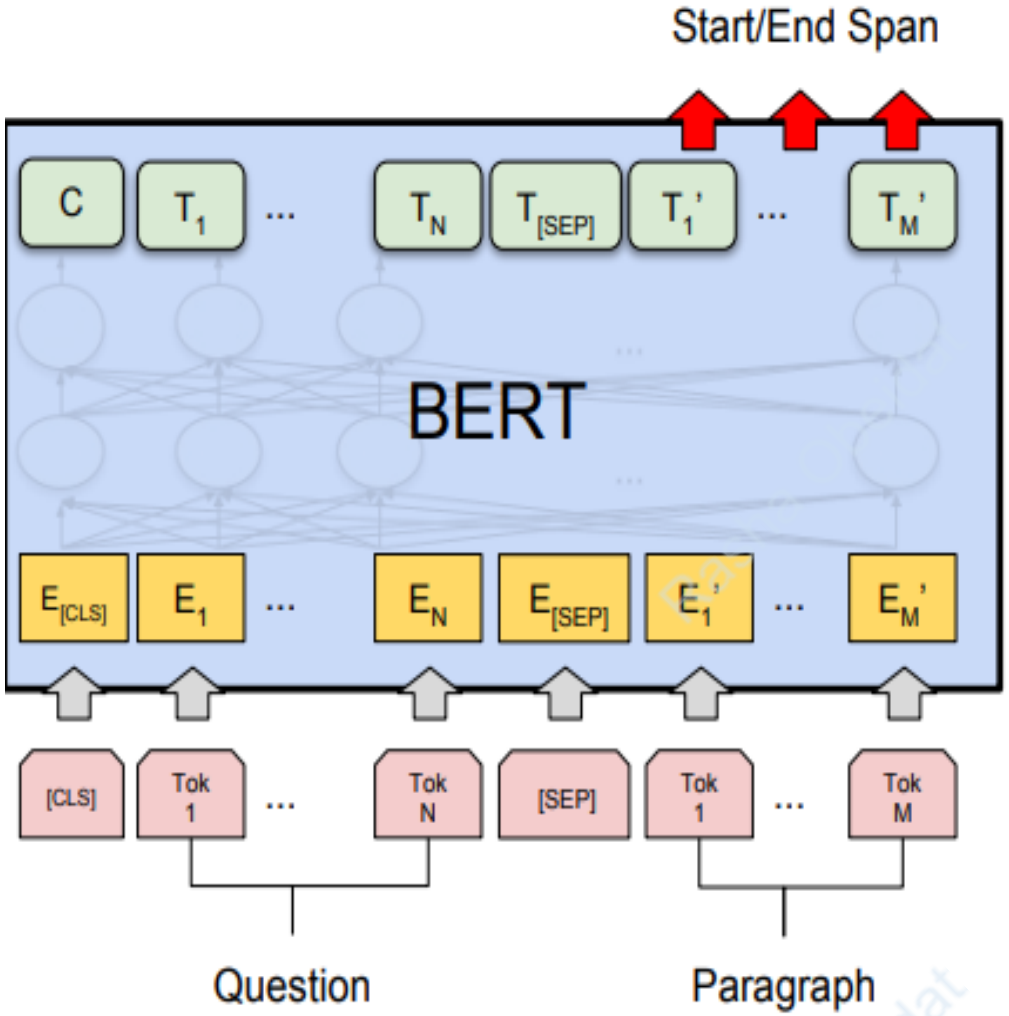
- BERT processes the question and the paragraph jointly as a single input, separated by the special [SEP] token.
- Each token receives a contextualized embedding that captures information from both the question and the paragraph.
- Instead of predicting a class label, BERT predicts two outputs for each token:
  - The probability of being the start of the answer span.
  - The probability of being the end of the answer span.
  - Start and end tokens together define the extracted answer span from the paragraph.



BERT Paper: https://aclanthology.org/N19-1423.pdf

Fine-tuning BERT for MRC

- After passing the whole input ([CLS] +question + [SEP] + paragraph [SEP]) through BERT's Transformer layers, we get a sequence of contextualized embeddings: $\boldsymbol{h_1}, \boldsymbol{h_2}, \ldots, \boldsymbol{h_l}$ where h_i is the final hidden vector for token i.
- Let $\boldsymbol{H} \in \boldsymbol{R^{l \times d}}$ be the matrix of final representations of the tokens of the paragraph. I.e., **H=** $[\boldsymbol{h_1}, \boldsymbol{h_2}, \ldots, \boldsymbol{h_m}]$ where $\boldsymbol{m}$ is the paragraph length and $\boldsymbol{d}$ is the dimensionality of the vectorized representation of each token produced by BERT (768 in original BERT).



BERT Paper: https://aclanthology.org/N19-1423.pdf

Fine-tuning BERT for MRC

- **Start** vector $S \in R^d$ and **End** vector $E \in R^d$ are introduced, The probability of token i being the start of the answer span is computed as a dot product between h_i and S followed by a Softmax over all of the tokens in the *paragraph*:

$$P_i = \sum_{i=1}^{m} \frac{e^{Sh_i}}{e^{Sh_j}}$$

- Likewise, the probability of token i being the start of the answer span is computed as a dot product between h_i and S followed by a Softmax over all of the tokens in the *paragraph*:

$$P_i = \sum_{i=1}^{m} \frac{e^{Eh_i}}{e^{Eh_j}}$$

Summary

BERT output → (512 tokens, 768 dim)

Linear layer for Start → (512 tokens, 1 score)

Linear layer for End → (512 tokens, 1 score)

Softmax over tokens → probability of each token being start or end

Fine-tuning BERT for MRC

- The, score of a candidate span from position i to position j is defined as $S \cdot T_i + E \cdot T_j$ and the maximum scoring span where $j \geq i$ is used as a prediction.
- During prediction, illogical spans where the start token comes after the end token are rejected. Only valid (start, end) pairs are considered.
- The training objective is the sum of the log-likelihoods of the correct start and end positions.

```
BERT output → (512 tokens, 768 dim)

Linear layer for Start → (512 tokens, 1 score)
Linear layer for End → (512 tokens, 1 score)

Mask out [CLS], question, and [SEP] tokens

Softmax over context tokens → probability of being start or end
```

Summary

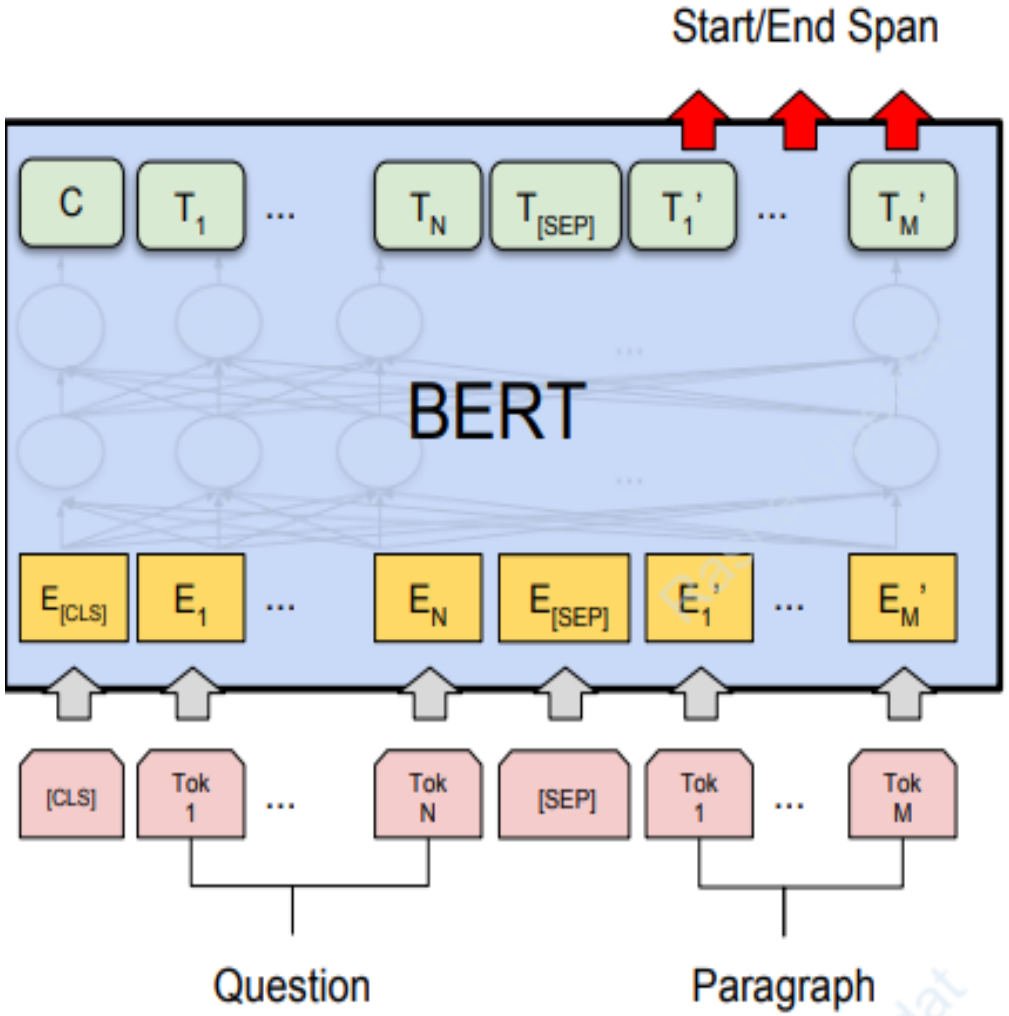
Fine-tuning BERT for MRC

- Example Dataset:
  - SQuAD (Stanford Question Answering Dataset)

Rajpurkar, Pranav, et al. "SQuAD: 100,000+ Questions for Machine Comprehension of Text." Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing. 2016.

  - ArQuad (Arabic Question Answering Dataset)

Obeidat, Rasha, et al. "ArQuAD: An Expert-Annotated Arabic Machine Reading Comprehension Dataset." Cognitive Computation 16.3 (2024): 984-1003.(https://github.com/RashaMObeidat/ArQuAD)

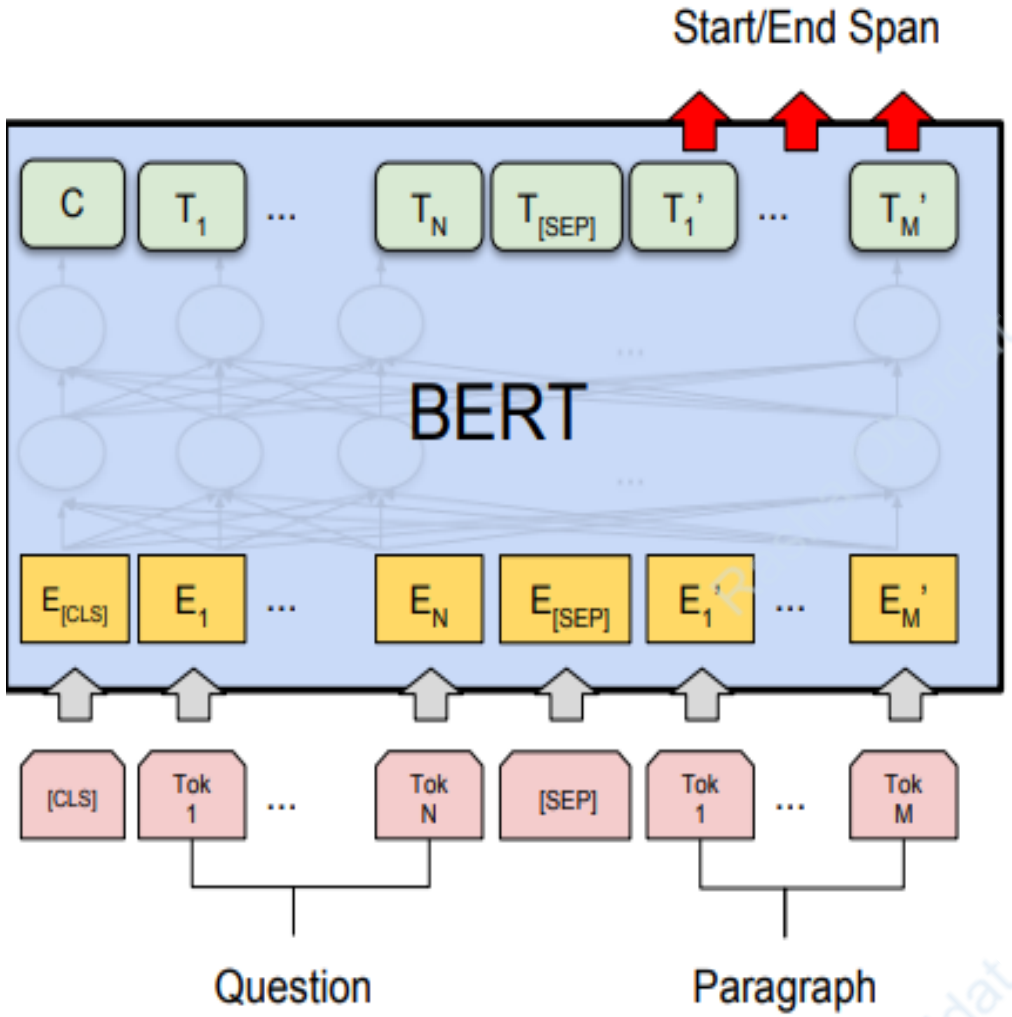


BERT Paper: https://aclanthology.org/N19-1423.pdf

Fine-tuning BERT for MRC

Table 2: SQuAD 1.1 results. The BERT ensemble is 7x systems which use different pre-training check-points and fine-tuning seeds.

System	Dev		Test	
	EM	F1	EM	F1
Top Leaderboard Systems (Dec 10th, 2018)				
Human	86.3	89.0	86.9	89.5
#1 Single - MIR-MRC (F-Net)	-	-	74.8	78.0
#2 Single - nlnet	-	-	74.2	77.1
Published				
unet (Ensemble)	-	-	71.4	74.9
SLQA+ (Single)	-		71.4	74.4
Ours				
$\text{BERT}_{\text{LARGE}}$ (Single)	78.7	81.9	80.0	83.1



BERT Paper: https://aclanthology.org/N19-1423.pdf

Which parts of BERT are Fine-tuned

During fine-tuning, **the entire BERT model is updated**, including:

- Token embeddings
- Segment embeddings
- Positional embeddings
- All transformer layers (attention and feedforward layers)

- If embeddings were **frozen**, the model would perform much worse.
- Fine-tuning embeddings is crucial because it allows BERT to ***adapt representations*** to the specific vocabulary and structure of the downstream task.

Which BERT Is Fine-tuned for Each Type of Task?

- What to tune?
  - During fine-tuning, the entire BERT model including all embeddings and transformer layers, is updated based on the downstream task supervision.
  - **If embeddings were frozen,** the model would perform much worse updating embeddings allows task-specific adaptation.
- Which BERT Is Fine-tuned for Each Type of Task?
  - For **single sentence classification**, fine-tuning adapts the model that was pretrained using **MLM**.
  - For **sentence pair classification tasks**, fine-tuning adapts the model that was pretrained with **both MLM and NSP** . NSP pretraining is particularly useful for tasks requiring reasoning between two sentences.
    - Pretraining using **both MLM and NSP means that they are** applied simultaneously on each batch
  - For **MRC** (such as ArQuAD), fine-tuning relies on the BERT representations learned through **MLM** pretraining. NSP is not needed, since the focus is on extracting answer spans within a single paragraph.
  - For **sentence tagging tasks**, fine-tuning updates the BERT model trained with **MLM** only, because tagging depends on rich token-level understanding, not sentence relationships.

Catastrophic Forgetting During BERT Fine-tuning

- It is the phenomenon where a neural network **forgets its previously learned knowledge** when fine-tuned on a new task.
- During fine-tuning, the model's weights are updated to fit the new task, possibly **overwriting important pretrained knowledge**.
- Leads to:
  - Loss of general language understanding learned during pretraining.
  - Overfitting to the fine-tuning dataset and consequently poor generalization to new, unseen examples.
  - Decreased performance on tasks requiring deep semantic understanding.

Catastrophic Forgetting: Causes and Mitigation

- **Reasons related to data:**
  - Fine-tuning dataset is **very small** → the model overfits quickly.
  - Fine-tuning task is **very different** from pretraining domain → big mismatch.
- **Reasons related to fine-tuning strategy:**
  - Large learning rate → aggressive weight updates.
  - Too many epochs → model over-specializes.
  - No regularization or constraints on the pretrained weights.
- **How to avoid it?**
  - Use a small learning rate (typically, less than 1e-3) ➔ Avoids aggressive weight updates that erase pretrained knowledge.
  - Fine-tune for a limited number of epochs (often 3–5 are enough) ➔ Prevents the model from over-specializing on the new task.
  - Choose fine-tuning tasks similar to pretraining data when possible

Catastrophic Forgetting: How to Avoid

- **Use a Small Learning Rate**
  - Fine-tuning should use a much smaller learning rate than pretraining.
  - This allows the model to **adapt gradually** without overwriting pretrained knowledge.
- **Freeze & Gradual Unfreezing**
  - Freeze lower transformer layers to preserve general language knowledge.
  - Train only the top layers / task head at the start of fine-tuning.
  - We can also *gradually* unfreeze layers (top → bottom) as training progresses
- **Use Regularization**
  - Techniques such as **dropout** and **weight decay** reduce overfitting.
  - Prevent the model from relying too heavily on fine-tuning data.
- **Use Sufficient and Diverse Fine-tuning Data**
  - Small datasets increase the risk of forgetting.
  - More data helps the model retain general language knowledge.

BERT Variants

- **ALBERT (A Lite BERT)**
  - Reduces model size by sharing parameters across layers and using factorized embeddings.
  - Faster and lighter, good for limited resources.
- **ELECTRA**
  - Trains more efficiently by replacing MLM with **Replaced Token Detection**.
  - The model learns to distinguish "real" vs "fake" tokens.
- **RoBERTa**
  - Improves BERT by removing NSP and training longer on more data.
  - Uses dynamic masking during training.
- **DistilBERT**
  - Compresses BERT into a smaller model by **knowledge distillation**.
  - 40% smaller, 60% faster, 97% of BERT's performance.

- These slides are the intellectual property of **Dr. Rasha Obeidat®**.
- The slides may **not be modified or redistributed** without explicit permission.
- **Contact:**
Obeidat.rm@gmail.com
Rmobeidat@just.edu.jo



Rasha Obeidat